



(/rol/app/)

Menu

Korean

홈 (/rol/app/) (home) > Managing Traffic with OpenShift Service Mesh

> Chapter 1: Managing Traffic with OpenShift Service Mesh

Managing Traffic with OpenShift Service Mesh

Lesson

Lab Environment

Resilience with OpenShift Service Mesh

Objectives

- Explain resilience strategies including load balancing, timeouts, retries, and circuit breakers.
- Configure connection pooling and outlier detection to prevent overload and automatically manage failing service replicas.
- Evaluate trade-offs when combining multiple resilience policies for defense-in-depth protection.

Resilience with OpenShift Service Mesh

Resilience is the ability of your service mesh to protect services from failures and improve fault tolerance. Red Hat OpenShift Service Mesh offers resilience features that enable your applications to operate reliably, ensuring the service mesh can tolerate failing nodes and prevent localized failures from cascading to other nodes.

Microservices-based architectures face the following reliability challenges from distributed computing:

Latency spikes

Network congestion or slow service responses can cause delays that propagate through your application, degrading the overall user experience.

Backend failures

Services might become temporarily unavailable due to crashes, restarts, or configuration issues. Without resilience strategies, these failures propagate to dependent services.

Network flakiness

Networks are inherently unreliable. Requests may be lost, dropped, or delayed due to network issues, resulting in communication failures between services.

Therefore, assume that applications will experience problems and outages.

Preparing for resilience is a way to make applications more reliable and ready to overcome these challenges. OpenShift Service Mesh includes resilience features as part of its traffic management model. Virtual services and destination rules resources enable you to configure flexible resilience strategies at different levels, such as the service level or the subset level.

NOTE

This lecture uses the following terminology:

Application

The software workload you deploy (for example, a web server or API).

Service

The Kubernetes Service resource that provides a stable network endpoint for your application.

Replica or Pod

Individual running instances of your application behind a service. These are the actual containers that process requests.

When Istio documentation refers to a "host" in the context of resilience policies, it means an individual replica or pod, not the Kubernetes Service. Istio applies resilience policies at the replica level.

Resilience Strategies

OpenShift Service Mesh provides the following resilience strategies:

Load balancing

Distributes load among service replicas to prevent overloads. Requires at least one redundant replica, the *N+1 rule*.

Retries

Automatically retries failed requests to handle transient failures, such as network outages or momentary overloads.

Time-outs

Prevents requests from hanging indefinitely by rejecting requests after a specified duration. Releases blocked resources and prevents cascading failures.

Circuit breakers

Stops sending traffic to failing or overloaded services. Configured statically, the *connection limits*, and dynamically, the *outlier detection*.

Connection pooling

Limits simultaneous connections to prevent overload. Configurable for both TCP connections and HTTP requests.

Outlier detection

Automatically identifies and temporarily removes failing instances from the load balancing pool.

Sticky sessions

Provides session affinity by directing requests from the same user to the same instance by using consistent hash-based load balancing.

Load Balancing

Load balancing distributes requests efficiently across service endpoints to prevent overload and improve reliability. When you distribute the load among multiple replicas of the same service, if one of the replicas experiences a failure, then the load balancer removes that replica from its pool and distributes the load among the healthy replicas.

To be resilient, the service must have at least one redundant replica, which is known as the $N+1$ redundancy rule. For example, if a service receives 50 requests per second and each replica can handle 10 requests per second, then you need five replicas to handle the load, plus one redundant replica, for a total of six. If one of the replicas fails, then the service still has five replicas that can handle 100% of the load.

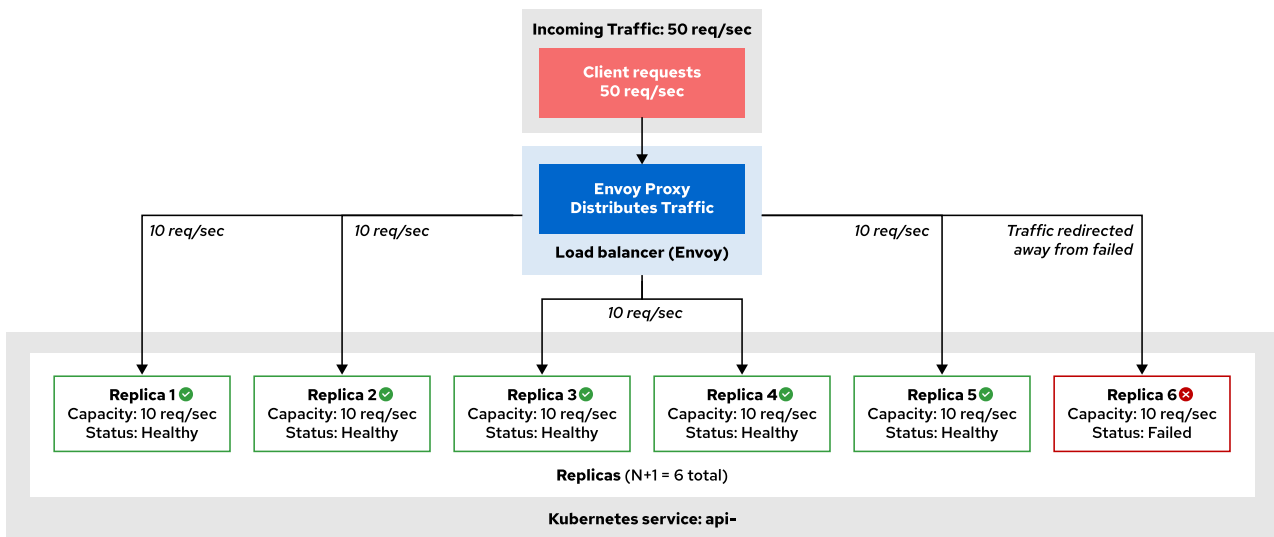


Figure 1.10: Load balancing with N+1 redundancy distributes traffic across six replicas

OpenShift Service Mesh provides several load-balancing algorithms to distribute requests across service instances. You configure load balancing at the service level by using the destination rule resource. Specifically, you set the value of the `spec.trafficPolicy.loadBalancer.simple` field to one of the following algorithms:

LEAST_REQUEST

The load balancer sends requests to endpoints with the fewest outstanding requests. Spreads load across endpoints, favoring those with the least active requests. Suitable for endpoints with uneven processing loads. Generally safer than ROUND_ROBIN and performs better in nearly all cases.

Use this algorithm when your service instances have different processing capabilities or when request processing times vary significantly.

NOTE

The LEAST_CONN algorithm is deprecated. Use LEAST_REQUEST instead.

ROUND_ROBIN

The load balancer sends requests to each replica in turn, distributing the load evenly. Use when all replicas have similar capabilities and consistent processing times.

RANDOM

The load balancer sends requests to replicas randomly. Under high loads, the load balancer distributes requests randomly across replicas. Helps avoid predictable request patterns. Use when you want to avoid potential issues with sequential routing patterns.

The following is an example of a destination rule that uses the least request load balancer:

```
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: my-destination-rule
spec:
  host: my-svc
  trafficPolicy: ❶
    loadBalancer:
      simple: LEAST_REQUEST ❷
```

❶ Traffic policy defined for the my-svc service.

❷ Load balancing algorithm name.

You can also define load balancing policies at the subset level, applying specific load balancers to different versions of the same service. The following example shows how to specify load-balancing features both at the service and the subset level for a specific version:

```
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: my-destination-rule
spec:
  host: my-svc
  trafficPolicy: ❶
    loadBalancer:
      simple: LEAST_REQUEST
  subsets:
  - name: v1
    labels:
      version: v1
  - name: v2
    labels:
      version: v2
    trafficPolicy: ❷
      loadBalancer:
        simple: RANDOM
```

- ❶ Traffic policy that applies the LEAST_REQUEST load balancer to the service.
- ❷ Traffic policy that applies the RANDOM load balancer to the v2 subset, overriding the policy at the service level.

For example, consider the following API backend service where different endpoints have significantly different processing costs:

- /health endpoint returns immediately (1ms)
- /search endpoint performs database queries (100ms)
- /report endpoint generates complex reports (5s)

When you use ROUND_ROBIN load balancing, a replica that receives several /report requests in succession becomes heavily loaded, whereas other replicas handling /health requests remain underutilized.

The LEAST_REQUEST load balancing strategy addresses this imbalance. The load balancer directs new requests to replicas with fewer outstanding requests. As a replica processes expensive /report requests, it accumulates active requests, and the load balancer automatically routes new requests to less busy replicas.

This ensures that all replicas maintain similar load levels, even when the costs of processing requests vary significantly.

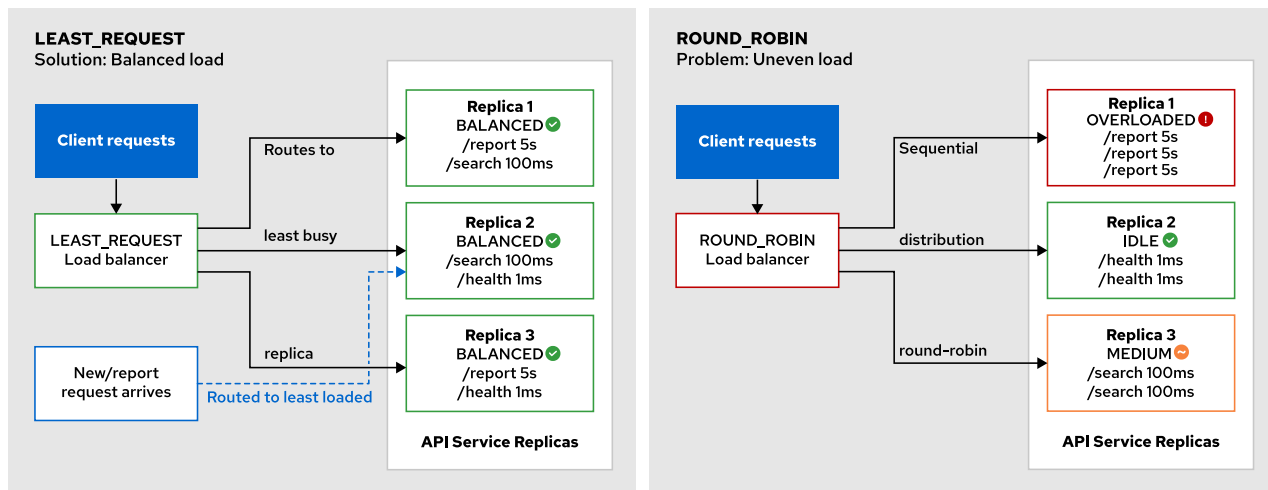


Figure 1.11: Comparison between LEAST_REQUEST and ROUND_ROBIN load-balancing strategies

Consistent Hash Load Balancing

Consistent hash-based load balancing provides session affinity by mapping HTTP headers, cookies, or source IP addresses to specific replicas. This strategy enables requests from the same user to consistently route to the same replica, maintaining session state.

OpenShift Service Mesh implements consistent hashing as soft session affinity, maintained on a best-effort basis. When replicas are added or removed from the service, affinity can be lost for a portion of requests. This differs from traditional sticky session implementations that encode specific destination information in cookies, providing stronger affinity guarantees as long as the replica remains available.

The following example configures consistent hash load balancing by using the `session_id` cookie as the hash key:

```
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: my-destination-rule
spec:
  host: my-svc
  trafficPolicy:
    loadBalancer:
      consistentHash:
        httpCookie:
          name: session_id ❶
          ttl: 0s ❷
```

❶ Cookie name to use as the hash key for routing decisions.

❷ Time-to-live for the cookie (0s means session cookie).

Retries

Services might be temporarily unavailable due to transient problems, such as network outages or momentary overloads. To address this situation, retrying the request with identical parameters might still succeed.

OpenShift Service Mesh enables you to implement the retry pattern without changing the application code. Consequently, you can change the retry configuration at runtime, without recompiling or redeploying your application.

In a cloud-native environment, microservices often rely on networks to communicate with other microservices. Because networks can be unreliable, a microservice might encounter issues during a communication request, such as:

- A request is lost, mishandled, or dropped due to an overloaded network.
- A target service experiences a temporary failure, for example, due to storage becoming temporarily disconnected.
- A subset of target service pods experiences a failure.
- A request response takes longer than expected, resulting in the source service experiencing a time-out.

When a request fails due to any of the errors described above, a repeated request with identical parameters might still succeed. The retry pattern prevents propagation of such transient errors into the application.

You can configure the retry pattern in the virtual service resource by using the `retries` field.

NOTE

Virtual services retry failed requests twice by default. The default retry conditions are: `connect-failure`, `refused-stream`, `unavailable`, and `cancelled`.

The following example configures retry behavior for the `example-svc` service:

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: example-vs
spec:
  hosts:
  - example-svc
  http:
  - route:
    - destination:
        host: example-svc
        subset: v1
    retries: 1
      attempts: 3
      perTryTimeout: 2s
      retryOn: gateway-error,connect-failure,refused-stream
```

- 1 The `retries` object responsible for configuring retry behavior.
- 2 The number of retry attempts. The maximum possible number of requests is 1 + attempts.
- 3 A time-out value for each retry attempt. Valid time units are `ms` (milliseconds), `s` (seconds), `m` (minutes), or `h` (hours).
- 4 A policy that specifies conditions that cause failed requests to retry. The value is a comma-separated list of retry policies.

To turn off the retry configuration, set the `attempts` parameter to 0:

The virtual service resource enables you to select one or more retry policies. The Envoy proxy evaluates each failed request by using the retry policies. If the Envoy proxy matches a request with any of the selected policies, then it retries that request.

Selecting retry policies is important for the optimal behavior of your service mesh. The following are examples of retry policies you might use with OpenShift Service Mesh:

gateway-error

Matches responses that contain the 502, 503, or 504 response codes. Use to retry requests that fail due to gateway or upstream service issues.

connect-failure

Matches requests that fail to connect to the upstream service. Use to handle network connectivity issues.

refused-stream

Matches requests where the upstream service actively refuses the connection. Use to handle cases where services are rejecting requests due to overload.

unavailable

Matches requests that receive a 503 (Service Unavailable) response code or gRPC UNAVAILABLE status. Use to retry requests when the upstream service is temporarily unavailable.

cancelled

Matches requests that are cancelled before completion, typically receiving a gRPC CANCELLED status. Use to retry requests that are terminated prematurely.

NOTE

Refer to the references section for complete retry policy documentation.

When choosing a retry policy, it is a good practice first to analyze all failed requests in your application and select the most specific policy for each case. For example, an application pod might take a long time to start and respond to the first requests with a timeout. You can mitigate the issue by using the `connect-failure` or `refused-stream` retry policy.

For example, consider an API service that occasionally returns gateway errors, such as 502, 503, or 504 status codes, during brief periods of high load. Without retries, these transient errors propagate to clients, degrading the user experience.

The following configuration automatically retries requests that fail with gateway errors:

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: api-vs
spec:
  hosts:
  - api-service
  http:
  - route:
    - destination:
        host: api-service
    retries:
      attempts: 3 ❶
      perTryTimeout: 2s ❷
      retryOn: gateway-error ❸
```

- ❶ Envoy makes up to 3 retry attempts (4 total requests: 1 initial + 3 retries).
- ❷ Each retry attempt has a 2-second timeout.
- ❸ If the first request fails with a gateway error, then Envoy automatically retries. If all attempts fail, then the error is returned to the client.

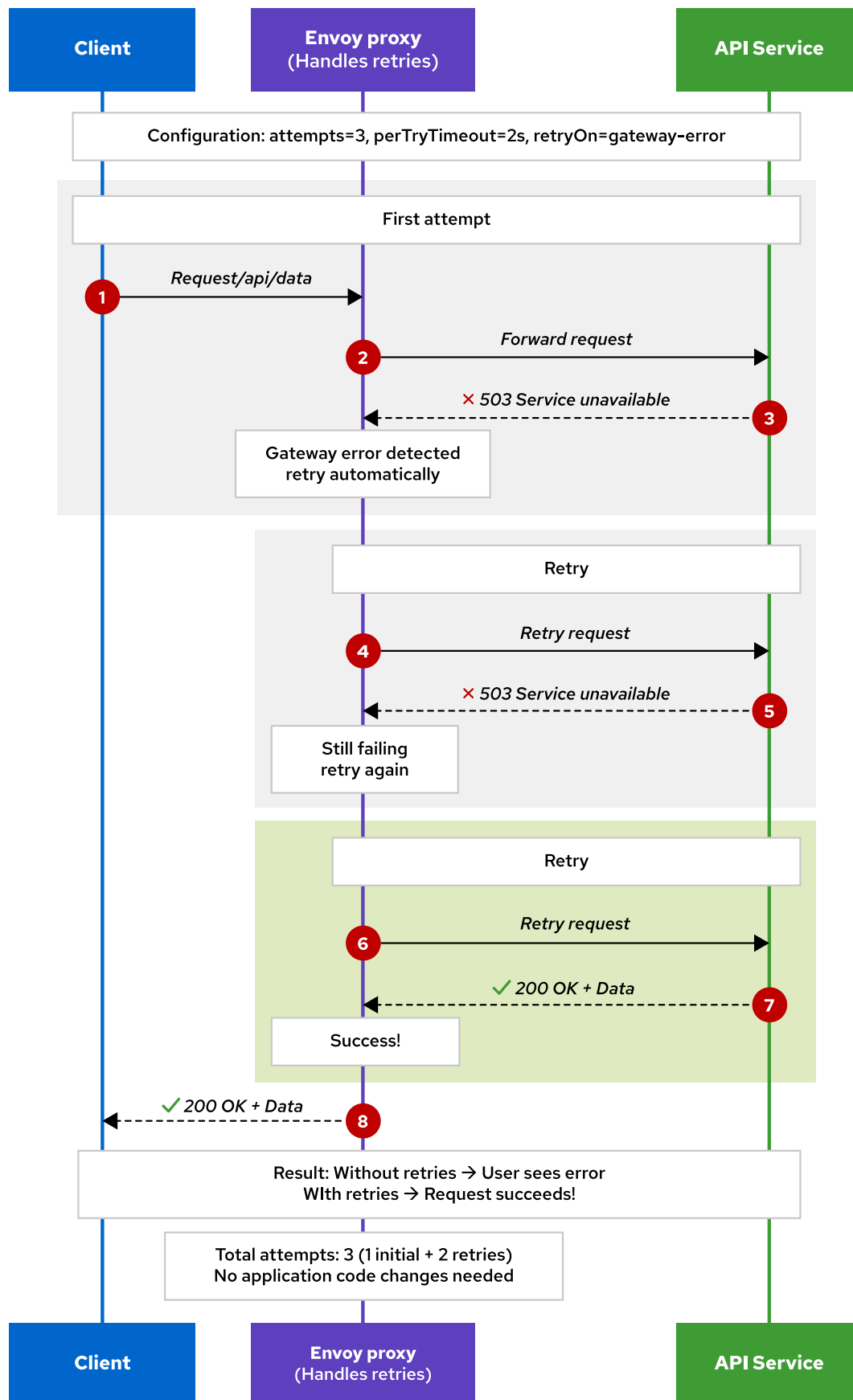


Figure 1.12: Retry pattern automatically retries failed requests

The configuration of the retry communication pattern requires careful consideration of the parameters of your environment, such as latency requirements, network layout, and application complexity.

There are no standard values for the retry configuration that are suitable for every environment. The following considerations can help when selecting retry parameters:

Impact on performance

Incorrect retry policies can substantially impact performance. Retries increase the volume of failed requests if the backend is misconfigured.

Number of attempts

More retries increase the success probability but also lead to network saturation. Balance attempts based on service failure patterns and capacity.

Timeout values

Longer timeouts help compute-intensive services but increase overall latency. Set `perTryTimeout` based on expected response times.

An incorrect retry setting makes your environment less resilient and can exacerbate performance issues within it. Other resilience patterns, such as circuit breakers, can mitigate worst-case retry scenarios. Additionally, Red Hat recommends implementing monitoring to alert you to possible issues as soon as they occur.

Time-outs

When requesting a service, you might encounter errors, such as slowdowns or network failures.

Instead of waiting indefinitely when these errors occur, establish a time-out after which the request is rejected. Setting a timeout helps applications release resources and prevent requests from hanging indefinitely, ensuring that network requests complete within a specified time. It also protects the whole system against cascading failures.

To improve the resilience of your applications, you can configure time-outs in OpenShift Service Mesh without modifying your application code.

Time-outs provide the following benefits:

Mitigating cascading failures

Because your application fails early, you prevent slow responses from downstream services from propagating to systems that depend on yours. Time-outs help contain failures and prevent them from affecting the entire system.

Guaranteed response time

A time-out ensures that a network request completes within a specified time limit. This predictability helps maintain consistent application performance.

Better resource usage

Time-outs reduce the time an application is blocked waiting for a response. This enables your application to release resources faster and handle other requests.

Virtual services enable you to configure time-outs for all traffic routed to a service. You can apply a time-out setting by using the `timeout` field in the route rules. The timeout value uses a duration format with units such as `s` (seconds), `ms` (milliseconds), `m` (minutes), or `h` (hours).

For example, consider an API service that queries a database. Under normal conditions, the service responds within 500ms. However, during database maintenance or periods of heavy load, some queries may take significantly longer or hang indefinitely. Without time-outs, these slow requests consume resources and degrade the overall system performance. Client applications might wait indefinitely for responses that never arrive.

The following configuration sets a 2-second timeout on all requests to the API service:

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: api-service-vs
spec:
  hosts:
  - api-service
  http:
  - route:
    - destination:
        host: api-service
      timeout: 2s ❶
```

❶ Configures a two-second timeout for all requests to the `api-service`.

With this configuration:

- Normal requests complete successfully within 2 seconds.
- Slow queries that exceed 2 seconds receive a time-out error.
- Resources are released after 2 seconds instead of waiting indefinitely.
- The system can handle the time-out error gracefully, perhaps returning a cached result or error message to the user.

Selecting Time-outs for Resilience

Each application is different, and the time required to generate a response depends on multiple factors, such as how busy the application is or if it calls external services. There is no standard way of calculating a precise value for the time-out, but several considerations can help when defining a time-out value:

Permit slow responses

The value must accommodate slow but legitimate responses to arrive. Set the time-out value based on the maximum acceptable response time for your service's slowest operations under normal conditions.

Stop waiting for unresponsive services

The value must stop waiting for a response that is not returned. Time-outs prevent resources from being tied up indefinitely when a service fails to respond.

Balance latency and reliability

A high value increases latency, especially in distributed systems. A low value might cause legitimate slow responses to fail. Balance these trade-offs based on your application's requirements.

Consider resource consumption

A high value potentially increases the computing resources required to wait for a dead service to respond. Time-outs help prevent resource exhaustion by releasing blocked connections.

Time-outs are not the only solution to increase the reliability of your applications. You can combine timeouts with other resilience strategies, such as retries or circuit breakers, for more robust error handling.

Circuit Breakers

Circuit breakers limit requests to unhealthy or overloaded replicas, preventing cascading failures. When replicas experience transient errors, those errors tend to occur continuously. Circuit breakers use this knowledge to avoid directing requests to failing replicas temporarily.

When a replica experiences repeated failures, the circuit breaker temporarily removes it from the load balancing pool. Requests are redirected to healthy replicas instead of waiting for the failing replica to respond. This ban is temporary, so the replica will receive new requests as soon as normal function is restored.

This behavior has two benefits:

- As requests do not reach the failing replicas, the service is more responsive, even if some replicas are slow.
- Failing replicas stop receiving requests for some time, enabling them to recover from overload and resolve pending requests.

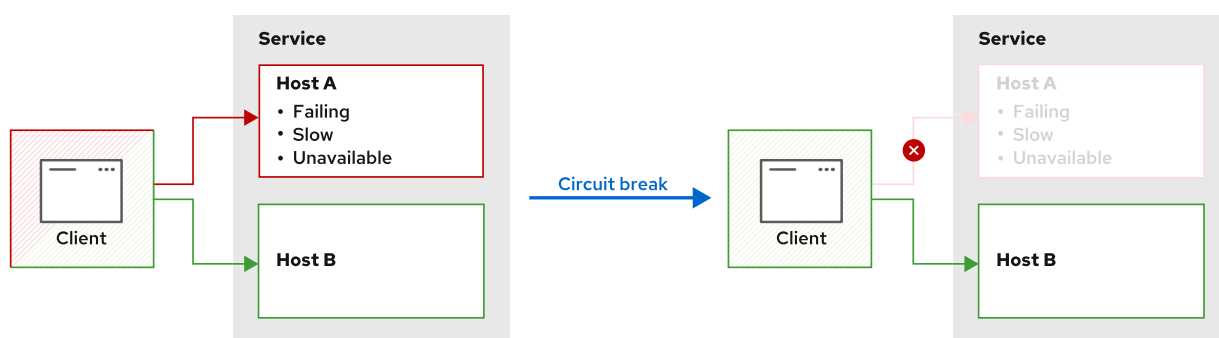


Figure 1.13: The circuit breaker

OpenShift Service Mesh implements circuit breakers at the replica level, not at the service level. This means OpenShift Service Mesh evicts failing replicas, not entire services or subsets. This behavior enables services to keep functioning even if some replicas fail.

For example, if you have a service with three pods and one of them begins to fail, then the eviction applies only to the failing pod. The service remains accessible and load balances requests among the two healthy pods.

Circuit breakers in OpenShift Service Mesh provide two complementary mechanisms: *connection pooling* and *outlier detection*.

Connection Pools

Limits the number of simultaneous connections and pending requests to prevent overload. If replicas are prone to time-out or fail when they receive too many requests, then limiting the number of connections helps prevent the replica from crashing.

You configure connection pools by using the `connectionPool` field in a destination rule resource. The following example shows a connection pool configuration:

```
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: my-destination-rule
spec:
  host: my-service
  trafficPolicy:
    connectionPool:
      tcp: ❶
        maxConnections: 100 ❷
        connectTimeout: 30ms ❸
      http: ❹
        http1MaxPendingRequests: 50 ❺
        maxRequestsPerConnection: 1 ❻
```

- ❶ TCP connection pool settings.
- ❷ Maximum number of simultaneous TCP connections per replica.
- ❸ Maximum time to establish a TCP connection.
- ❹ HTTP connection pool settings.
- ❺ Maximum number of HTTP requests pending per replica.
- ❻ Maximum number of requests permitted on a single connection.

Connection pools apply independently to each replica. If a replica depletes its connection pool, then OpenShift Service Mesh establishes no further connections to that replica but continues to use other replicas.

Outlier Detection

Outlier detection automatically identifies and temporarily removes failing replicas from the load balancing pool. This enables failing replicas time to recover without impacting user requests.

To enable outlier detection, include an `outlierDetection` entry in the destination rule resource:

```
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: my-destination-rule
spec:
  host: my-service
  trafficPolicy:
    outlierDetection:
      consecutive5xxErrors: 5 1
      interval: 10s 2
      baseEjectionTime: 30s 3
      maxEjectionPercent: 50 4
```

- ¹ Number of consecutive 5xx errors before evicting the replica.
- ² Time interval between checking error counts.
- ³ Minimum duration of replica ejection time.
- ⁴ Maximum percentage of evicted replicas for a service at any time.

The `baseEjectionTime` indicates the minimum ejection time. Subsequent evictions multiply this value by the number of times the replica is evicted. For example, if `baseEjectionTime` is 30 seconds:

- First eviction: 30 seconds
- Second eviction: 60 seconds
- Third eviction: 90 seconds

The `maxEjectionPercent` limits the percentage of replicas that can be evicted simultaneously, preventing all replicas from being evicted. Default is 10%.

For example, consider a backend service with three replicas that processes CPU-intensive tasks. Each replica can handle approximately 50 concurrent connections efficiently. Beyond that limit, response times degrade significantly and errors increase.

Without circuit breakers, clients continue sending requests to overloaded replicas, causing cascading failures.

The following configuration protects the backend service by using both connection pooling and outlier detection:

```

apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: backend-circuit-breaker
spec:
  host: backend-service
  trafficPolicy:
    connectionPool:
      tcp:
        maxConnections: 50 ❶
      http:
        http1MaxPendingRequests: 10 ❷
        maxRequestsPerConnection: 2 ❸
    outlierDetection:
      consecutive5xxErrors: 3 ❹
      interval: 5s ❺
      baseEjectionTime: 30s ❻
      maxEjectionPercent: 50 ❼

```

- ❶ Maximum of 50 TCP connections per replica.
- ❷ Maximum of 10 pending HTTP requests per replica.
- ❸ Maximum of 2 requests per connection before the connection is closed.
- ❹ Eject replica after 3 consecutive 5xx errors.
- ❺ Verify error counts every 5 seconds.
- ❻ Minimum ejection time of 30 seconds (increases with subsequent ejections).
- ❼ Maximum 50% of replicas can be ejected simultaneously.

Selecting Circuit Breakers for Resilience

Circuit breakers are useful to protect services prone to transient failures. Consider implementing circuit breakers in these scenarios:

Compute-intensive services

Services that receive more requests than they can respond to often experience transient failures. Circuit breakers redirect requests away from overloaded replicas, giving them time to recover from increased load.

Services with sequential request processing

Services that process requests sequentially usually store pending requests in a queue. If this queue becomes too large, then replicas take too much time to respond, degrading service quality. Circuit breakers detect these time-outs and give replicas time to empty the queue.

Services with resource constraints

Services with limited database connections, memory, or other resources benefit from connection limits. Circuit breakers prevent resource exhaustion by limiting the number of concurrent connections.

Combining Resilience Policies

OpenShift Service Mesh enables you to combine multiple resilience policies to provide defense in depth, but it requires careful consideration.

Retries with time-outs

Time-outs prevent requests from hanging indefinitely, whereas retries handle transient failures. Together, they ensure failed requests are completed quickly and can recover from temporary issues.

Circuit breakers with load balancing

Load balancing distributes traffic across healthy replicas, whereas circuit breakers automatically remove failing replicas from the pool. Together, they ensure traffic only reaches healthy replicas and enable failing replicas time to recover.

Connection pooling with outlier detection

Connection pooling limits concurrent connections to prevent overload, whereas outlier detection removes replicas that become unhealthy. Together, they prevent both overload and cascading failures.

Cautions When Combining Policies

Some policy combinations can amplify problems or create unexpected behavior:

Retries with aggressive time-outs

Aggressive time-outs, combined with numerous retry attempts, can amplify the load on struggling services. Each timeout triggers a retry, potentially overwhelming the service.

Balance the `timeout` value with the `attempts` count to avoid generating excessive load.

Sticky sessions with circuit breakers

Consistent sticky sessions can concentrate traffic on specific replicas. If a replica becomes overloaded and is ejected, then all users mapped to that replica experience disruption.

Use sticky sessions sparingly, and ensure adequate capacity on each replica to handle its share of the traffic. Monitor ejection rates to detect when sticky sessions cause imbalanced load.

High retry counts without circuit breakers

Many retry attempts during widespread failures create retry storms. Combine retries with circuit breakers to limit retry attempts to healthy replicas.

Use `maxEjectionPercent` to ensure some replicas remain available.

The following example demonstrates a comprehensive resilience strategy combining multiple policies:

```

apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: api-service-vs
spec:
  hosts:
  - api-service
  http:
  - route:
    - destination:
        host: api-service
      timeout: 5s ❶
      retries: ❷
        attempts: 2
        perTryTimeout: 2s
        retryOn: gateway-error,connect-failure

```

```

---
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: api-service-dr
spec:
  host: api-service
  trafficPolicy:
    loadBalancer: ❸
      simple: LEAST_REQUEST
    connectionPool: ❹
      tcp:
        maxConnections: 100
      http:
        http1MaxPendingRequests: 20
        maxRequestsPerConnection: 5
    outlierDetection: ❺
      consecutive5xxErrors: 3
      interval: 10s
      baseEjectionTime: 30s
      maxEjectionPercent: 50

```

- ❶ Overall timeout of 5 seconds for the entire request, including all retries.
- ❷ Up to 2 retry attempts, each with a 2-second timeout. Total possible time: 5 seconds (limited by the overall timeout).
- ❸ Distribute requests to the least-loaded replicas.
- ❹ Limit connections to prevent overload. Up to 100 concurrent connections per replica, with a maximum of 20 pending requests.
- ❺ Eject replicas that return three consecutive 5xx errors. Ejected replicas remain out of the load-balancing pool for at least 30 seconds. A maximum of 50% of replicas can be ejected simultaneously.

With this configuration:

- Normal requests complete within 5 seconds total.

- Transient failures, such as *gateway-errors* and *connection-failures*, are retried up to 2 times, each with a 2-second timeout.
- Requests are distributed to the least-loaded healthy replicas.
- Replicas that become overloaded by exceeding connection limits or unhealthy due to consecutive errors are automatically removed from the pool.
- The service remains available as long as at least 50% of replicas are healthy.

Guidelines for Combining Policies

When combining resilience policies, consider these guidelines:

Start with load balancing and monitoring

Before adding other resilience policies, ensure you have proper load balancing and monitoring in place. This foundation helps you understand normal service behavior and identify when problems occur.

Add time-outs next

Time-outs provide a safety net that prevents requests from hanging indefinitely. Set time-outs based on your service's expected response times under normal load.

Add retries cautiously

Configure retries only for transient, recoverable failures. Use specific retry policies (*gateway-error*, *connect-failure*) rather than broad policies (*5xx*). Limit retry attempts to avoid amplifying load during widespread failures.

Add circuit breakers for protection

Circuit breakers provide the final layer of protection, preventing overload and cascading failures. Use both connection pooling for static limits and outlier detection for dynamic ejection for comprehensive protection.

Test combined policies

Use chaos engineering techniques to validate that your resilience policies work as expected. Simulate failures and verify that policies activate correctly without creating new problems.

Monitor policy effectiveness

Track key metrics, including retry rates, timeout rates, and circuit breaker ejections. High rates might indicate configuration problems or underlying service issues that need attention.

REFERENCES

Istio - Traffic Management Concepts

(<https://istio.io/v1.26/docs/concepts/traffic-management/>)

Istio - VirtualService Reference

(<https://istio.io/v1.26/docs/reference/config/networking/virtual-service/>)

Istio - DestinationRule Reference

(<https://istio.io/v1.26/docs/reference/config/networking/destination-rule/>)

Istio - Traffic Management Best Practices

(<https://istio.io/v1.26/docs/ops/best-practices/traffic-management/>)

Envoy - Load Balancing Overview

(https://www.envoyproxy.io/docs/envoy/v1.34.2/intro/arch_overview/upstream/load_balancing/overview)

Envoy - Retry Policies

(https://www.envoyproxy.io/docs/envoy/v1.34.2/configuration/http/http_filters/router_filter#x-envoy-retry-on)

Envoy - Circuit Breaking

(https://www.envoyproxy.io/docs/envoy/v1.34.2/intro/arch_overview/upstream/circuit_breaking)

Envoy - Outlier Detection

(https://www.envoyproxy.io/docs/envoy/v1.34.2/intro/arch_overview/upstream/outlier)

Revision: ad00081-3.1-ed4-20260310-8ae9760



개인 정보 취급 방침 (https://www.redhat.com/ko/about/privacy-policy?extIdCarryOver=true&sc_cid=701f20000001D8QoAAK)

이용 약관 (<https://www.redhat.com/ko/about/terms-use>)

릴리스 노트

(https://docs.redhat.com/en/documentation/red_hat_learning_subscription/1-latest/html-single/red_hat_learning_subscription_release_notes/index)

Red Hat 교육 정책

(<https://www.redhat.com/ko/about/red-hat-training-policies>)

모든 정책 및 지침

(<https://www.redhat.com/ko/about/all-policies-guidelines>)

쿠키 기본설정